2조

김영채
박채린
전승현
김민서

부동산 가격 예측 :

www.miri_portfolio.com

Index :

1

주제 선정

1) 주제 변경

2) 변경된 가설

3) 실험 흐름 (모식도)

2

이론

1) 시계열 data 모델

2) ARIMA

3

코드 설명

1) 실험

4

가설 입증 및 결론

1) 분석

주제 선정 :

1-1. 주제 변경

초기 주제

> 국내 부동산의 해외자본 간섭율을 파악하여 실질적인 국내 부동산 매매율을 알아본다.

주제 선정 이유

> 데이터를 기반으로 최근 사회적으로 이슈가 되었던 부동산 매매가 상승률에 대한 가설을 입증하기 위해

주제 선정 :

1-1. 주제 변경

가설 > 큰 폭의 부동산 상승률은 해외자본의 영향이 있었을 것이다.

1단계
: 부동산 상승률 알아보기

2단계
: 물가 상승률과 부동산 상승률과 취합하여 실질적인 부동산 매매가 동향 알아보기(물가 거품 제거)

3단계
: 부동산에 한정하여 국내 해외 자본 유입율을 분석하기

4단계
: 2단계에서 가공되어 나온 데이터에 ~~3단계 데이터를 취합~~하여 부동산 실매매가를 측정하고 이를 통해 가설 입증하기. ~~(해외자본 거품 제거)~~

* 외국인 직접투자 동향과 외국인 아파트 매매율을 교차 검증하여 적절한 가중치를 부여하여 해외자본 투입율을 결정한다.

주제 선정 :

1-1. 주제 변경 사유

데이터 양의 부족

문제점 :

해외 외국인 자본이 두각된지 오래되지 않았기 때문에, 최소 2006년도 이상의 신뢰 데이터를 찾기 어려움.

-> 해외자본의 유입에 대한 가설을 입증하는 것보다 부동산 가격의 변동률을 예측하는 것으로 변경.

주제 선정

2-1. 데이터 가공 방향

1. 물가 데이터 가공 방법

$\frac{(\text{공업제품} + \text{서비스})}{2} + \text{공업제품}$	서비스 집세 + 공공서비스+개인서비스

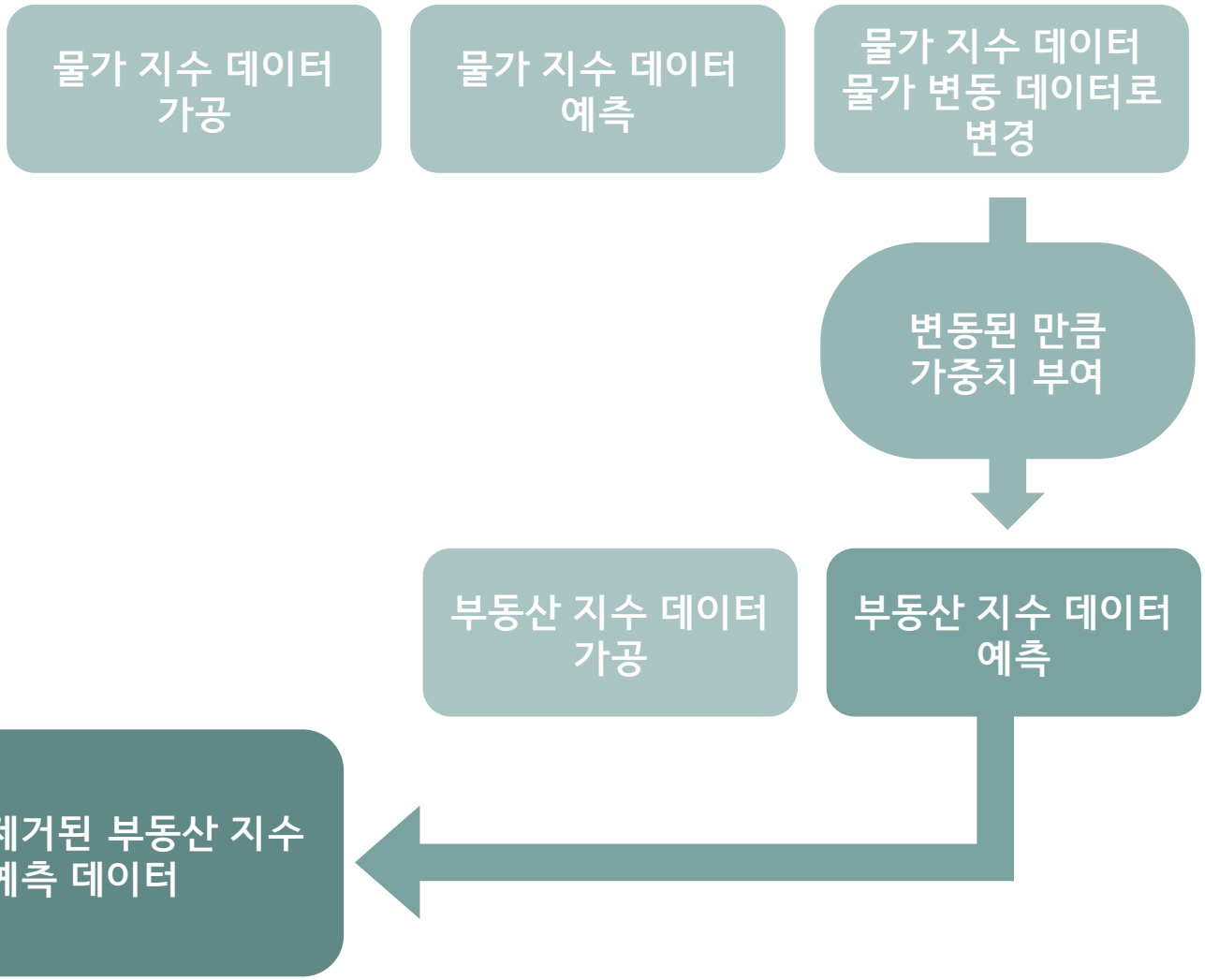
2. 부동산 데이터 가공 방법

- 지수와 변동률을 따로 데이터를 분리하기
- 지수랑 변동률 관계
  - 변동률 구하는 방법

$$V_n = k_n - k_{n-1}$$

주제 선정

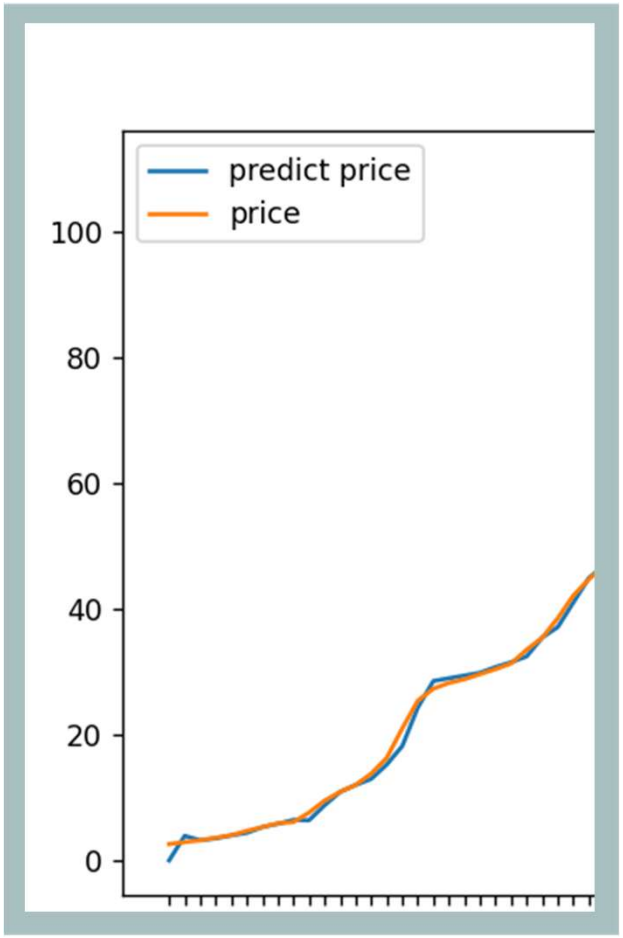
1-3. 실험 전개도



예측 범위 : ~2025년

이론

2-1. 시계열 데이터 모델



이론 :

2-2 ARIMA

ARIMA Model

: AR, MA 모델을 합친 것으로 현재 시점의 상태를 파악하는 과거 시점의 자기 자신과 추세까지 전부 반영하겠다는 의미

→ non-stationary한 상태에서 예측하는 모델을 말한다.

$$X(t) = w_{1,1} * X(t-1) + w_{2,1} * e(t-1) + b + u + e(t)$$

* non-stationary : 데이터와 시간에 따라 통계적 특성이 변화하는 비정상성

이론 :

2-2 ARIMA

AR Model (Autoregressive)

: 자기 회귀 모델

→ 과거의 시점의 자신의 데이터가 현 시점의 자기 자신에게 영향을 미치는 모델

$$X(t) = w * X(t-1) + b + u * e(t)$$

↳ 추세가 변하는 상황에서는 적합하지 않은 모델이다.

MA Model (Moving avarage)

: 이동 평균 모델

→ 트렌드 즉 추세가 변하는 상황에서 적합한 모델

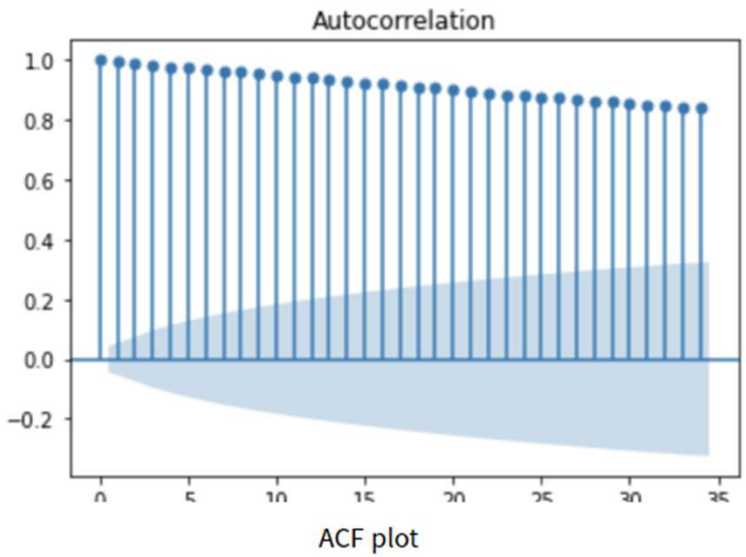
$$\mathrm{X(t)} = \mathrm{w} * \mathrm{e(t-1)} + \mathrm{b} + \mathrm{u} * \mathrm{e(t)}$$

↳ 변화율(추세)에 맞춰 수정하겠다는 의미다.

이론:

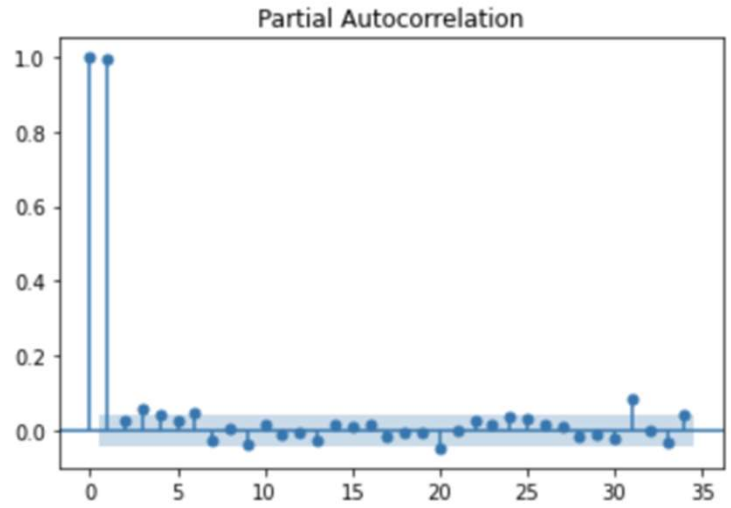
2-1. 시계열 데이터 모델

ACF



ACF plot

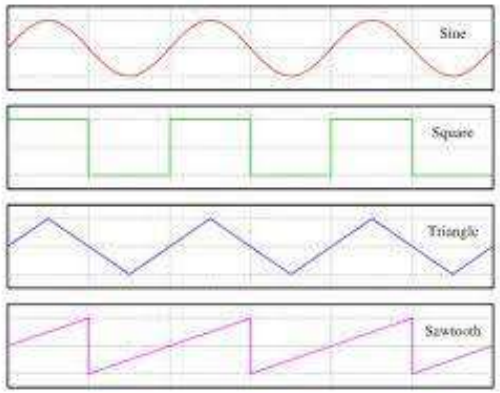
PACF

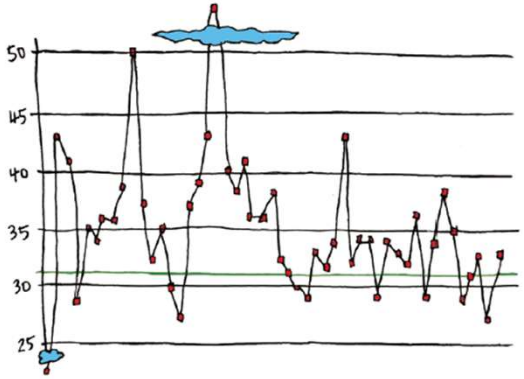


PACF plot

자기상관(autocorrelation)

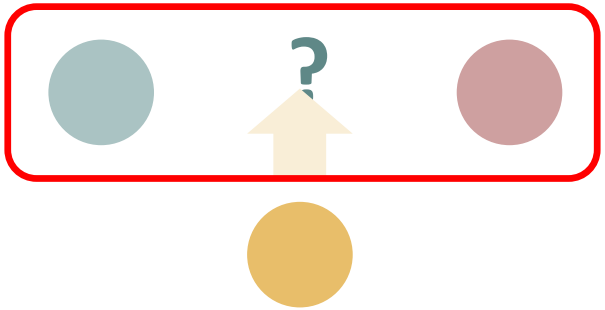
시간 또는 공간적으로 연속된 일련의 관측치들 간에 존재하는 상관관계





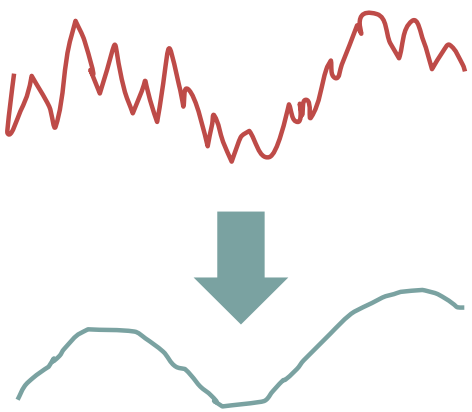
편자기상관(partial autocorrelation)

어떤 것들에 대한 상관관계를 보려고 할 때, 그에 영향을 주는 다른 요소들을 제외하는 상관관계



이론:

2-2 ARIMA



차분 = (현재 상태) – (바로 이전 상태)

↳ 차분을 걸친 결과들은 whitening 되는 효과가 있음

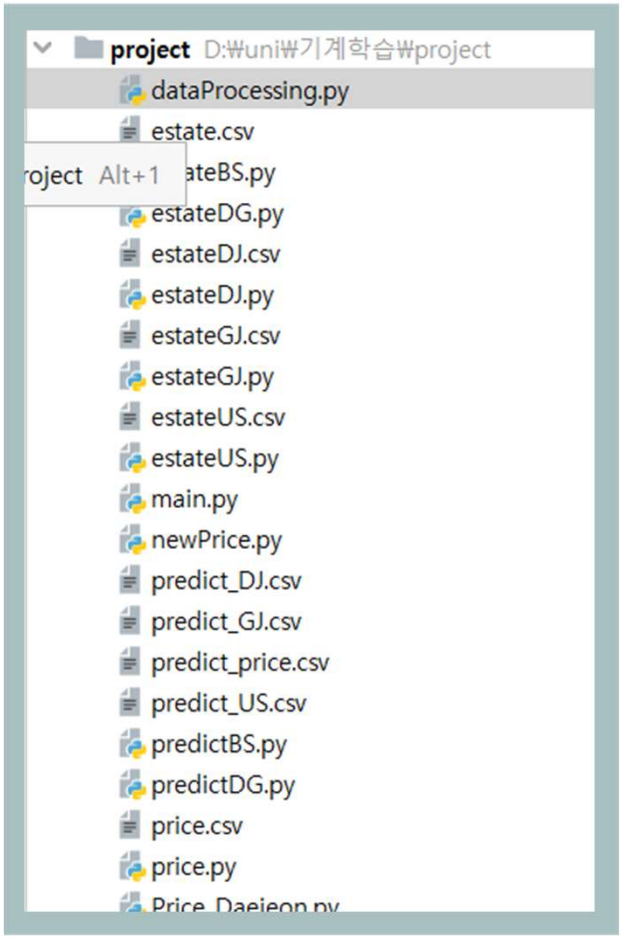
$$d = 0 : x_t = X_t$$

$$d = 1 : x_t = X_t - X_{t-1}$$

$$d = 2 : x_t = (X_t - X_{t-1}) - (Xt - 1 - Xt - 2)$$

코드 설명

3-1 실험


project D:₩uni₩기계학습₩project
dataProcessing.py
estate.csv
roject Alt+1
ateBS.py
estateDG.py
estateDJ.csv
estateDJ.py
estateGJ.csv
estateGJ.py
estateUS.csv
estateUS.py
main.py
newPrice.py
predict_DJ.csv
predict_GJ.csv
predict_price.csv
predict_US.csv
predictBS.py
predictDG.py
price.csv
price.py

물가 데이터 가공

dataProcessing.py

```python
#첫번째 실행 파일
#원형데이터인 priceData.csv가 필요함.
import numpy as np
import pandas as pd
import seaborn as sns
import warnings

np.set_printoptions(formatter={'float_kind':'{:f}'.format})
sns.set(rc={'figure.figsize':(8,6)})

warnings.filterwarnings("ignore")
warnings.filterwarnings(action='ignore')

data = pd.read_csv("./priceData.csv")   #데이터 삽입

data=data.transpose()   #행 열 전환
data.rename(columns=data.iloc[0], inplace=True)

#null데이터는 버릴거임. ,,,,이 부분들임.
data = data.drop(data.index[0])
data = data.drop(data.index[57:]) #인덱싱할 숫자 값은 보유한 데이터 규격에 맞춰서. 숫자는 엑셀에서 파일 오픈했을 때 마지막 열숫자-1

#csv 데이터의 column name이 전부 영어명이어야함.
#열 하나씩 슬라이싱
price_data = data[['price']]

#인덱스의 값을 열로 넣을 거임. (년도)
#년도는 어차피 모든 데이터가 똑같이 공유하기 때문에 한개만 생성해도 ok
y = price_data.index.values  #csv에서 인덱스 값을 y에 저장
price_data.insert(0, column='years', value=y) #0번째에 years라는 열이름으로 y삽입

# ##############################데이터가공
#슬라이싱한 열을 각각 하나의 파일로 저장.
price_data.to_csv('price.csv')
```

```
years,1965-01-01,1966-01-01,1967-01-01,1968-01-01,1969-01-01,1970-01-01,1971-01-01,1972-01-01,1973-01-01,1974-01-01,197
price,2.6,2.9,3.2,3.6,4,4.7,5.3,5.9,6.1,7.6,9.5,10.9,12,13.8,16.3,21,25.4,27.3,28.2,28.8,29.6,30.4,31.3,33.5,35.4,38.5,
variance,0,0.3,0.3,0.4,0.4,0.7,0.6,0.6,0.2,1.5,1.9,1.4,1.1,1.8,2.5,4.7,4.4,1.9,0.9,0.6,0.8,0.8,0.9,2.2,1.9,3.1,3.6,2.6,
```

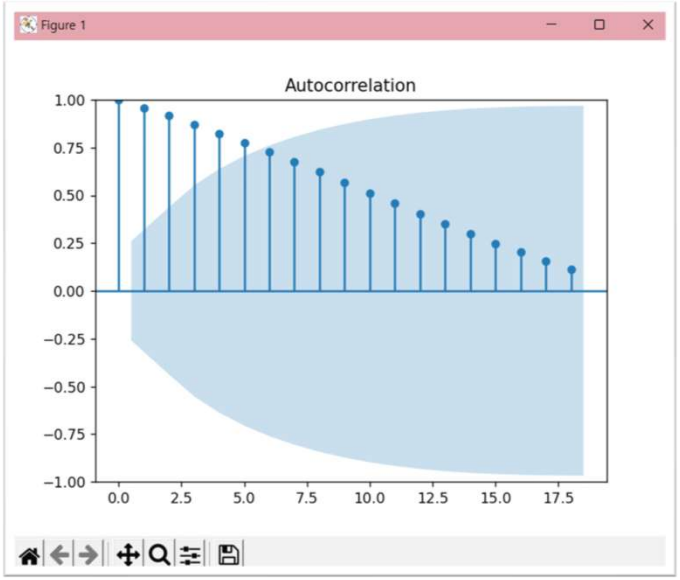
*.csv files are supported in other JetBrains IDEs

```
1994-01-01,1994-01-01,49.8
1995-01-01,1995-01-01,52
1996-01-01,1996-01-01,54.6
1997-01-01,1997-01-01,57
1998-01-01,1998-01-01,61.3
1999-01-01,1999-01-01,61.8
2000-01-01,2000-01-01,63.2
2001-01-01,2001-01-01,65.7
2002-01-01,2002-01-01,67.5
2003-01-01,2003-01-01,69.9
2004-01-01,2004-01-01,72.4
2005-01-01,2005-01-01,74.4
2006-01-01,2006-01-01,76.1
2007-01-01,2007-01-01,78
2008-01-01,2008-01-01,81.7
2009-01-01,2009-01-01,83.9
2010-01-01,2010-01-01,86.4
2011-01-01,2011-01-01,89.9
2012-01-01,2012-01-01,91.8
2013-01-01,2013-01-01,93
2014-01-01,2014-01-01,94.2
2015-01-01,2015-01-01,94.9
2016-01-01,2016-01-01,95.8
2017-01-01,2017-01-01,97.6
2018-01-01,2018-01-01,99.1
2019-01-01,2019-01-01,99.5
2020-01-01,2020-01-01,100
2021-01-01,2021-01-01,102.5
```

물가 데이터 예측

dataProcessing.py | estateBS.py | newPrice.py

```python
###두번째로 돌려봐야할 파일
###물가지수파일인 price.csv가 프로젝트 내에 있어야 실행됨

import pandas as pd
import matplotlib.pyplot as plt

df=pd.read_csv('./price.csv', header=0, names=['years','price']) #헤더가 0이면 첫번째 열을 이름으로 읽음
print(f"Total samples length : {len(df)}")

df['years']=pd.to_datetime(df['years'])  #년도의 string형을 timestamp형으로 바꾸어 저장.

from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

plot_acf(df.price)#레퍼런스코드

#차분
f=plt.figure()
ax1=f.add_subplot(1,2,1)
ax1.set_title('1st dif')
dif_1=ax1.plot(df.price.diff())

ax2=f.add_subplot(1,2,2)
plot_acf(df.price.diff().dropna(), ax=ax2)
plt.show()

#차분 2번째
f2=plt.figure()
ax3=f2.add_subplot(1,2,1)
ax3.set_title('2nd dif')
ax3.plot(df.price.diff().diff())

ax4=f2.add_subplot(1,2,2)
plot_acf(df.price.diff().diff().dropna(), ax=ax4) #dropna로 값이 비어있는 곳 삭제.
plt.show()

#차분 3번째
f3=plt.figure()
ax5=f3.add_subplot(1,2,1)
ax5.set_title('3rd dif')
ax5.plot(df.price.diff().diff().diff())

ax6=f3.add_subplot(1,2,2)
plot_acf(df.price.diff().diff().diff().dropna(), ax=ax6)
plt.show()

from statsmodels.tsa.stattools import adfuller
result=adfuller(df.price.dropna())
print('p-value : ', result[1])

result_dif1=adfuller(df.price.diff().dropna())
print('차분 1번한 p-value : ', result_dif1[1])

result_dif2=adfuller(df.price.diff().diff().dropna())
```




```
p-value :  0.9775307152065008
차분 1번한 p-value :  0.0520658865143812
차분 2번한 p-value :  0.011796310632963726
차분 3번한 p-value :  2.6066294424936302e-05
```

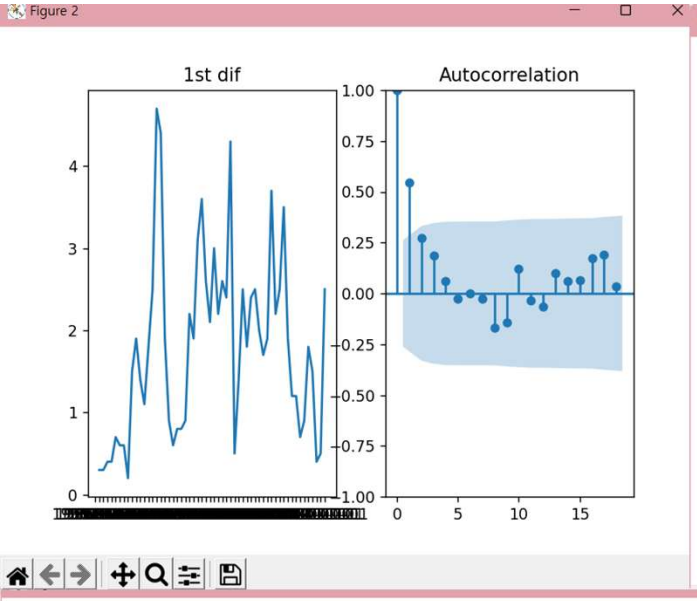


Figure 2
1st dif
Autocorrelation

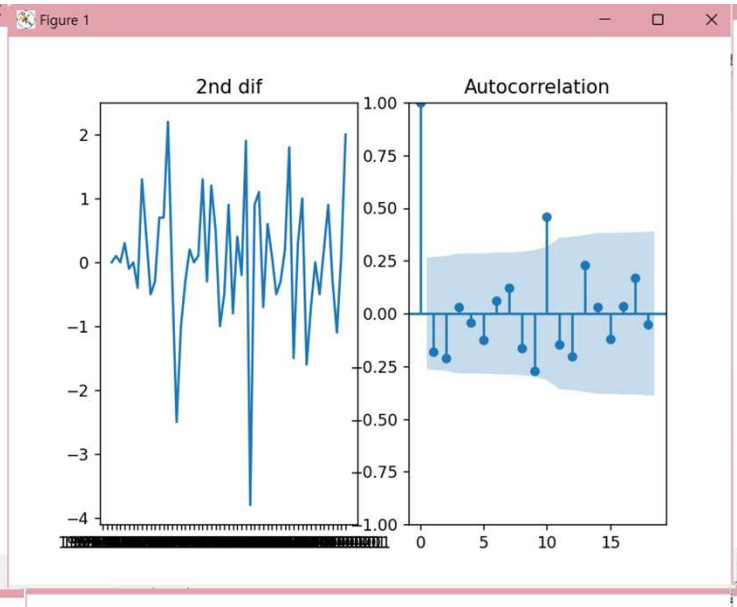
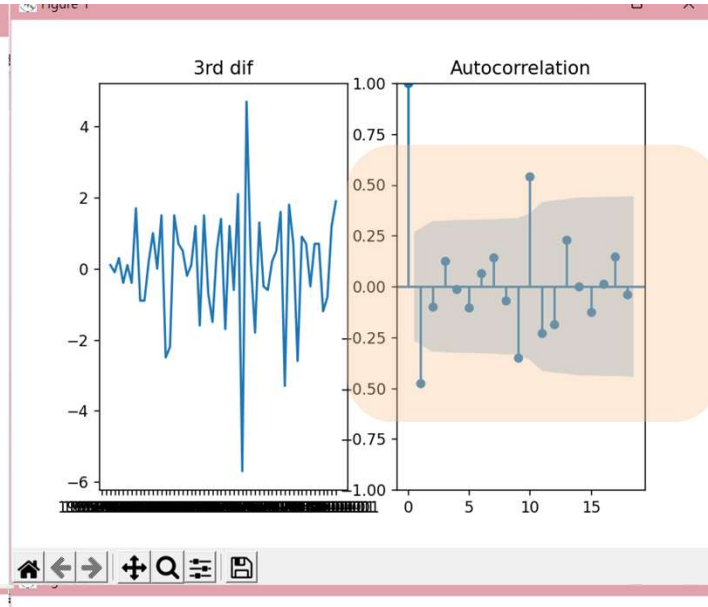
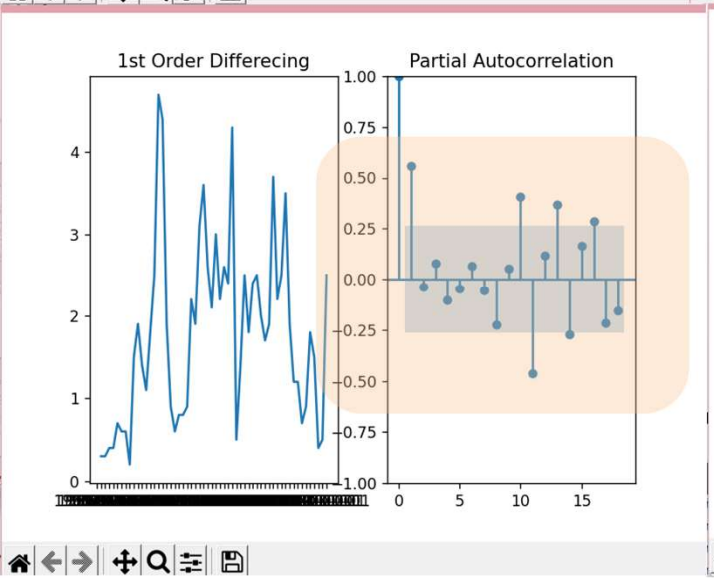
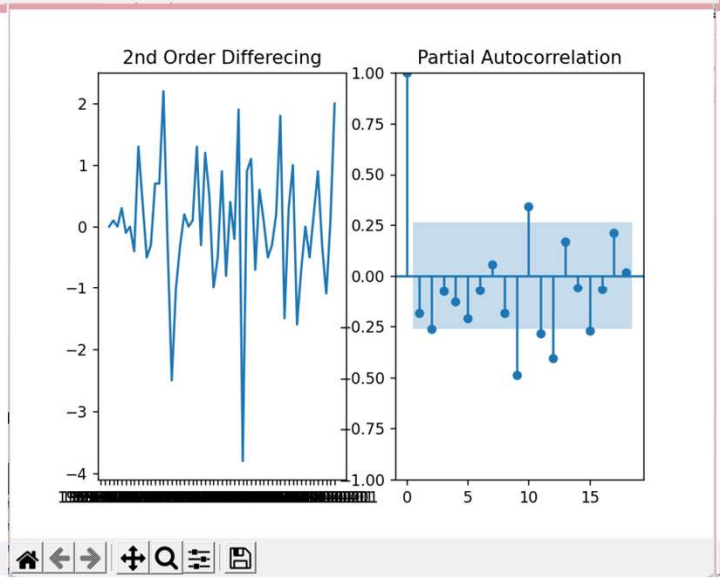
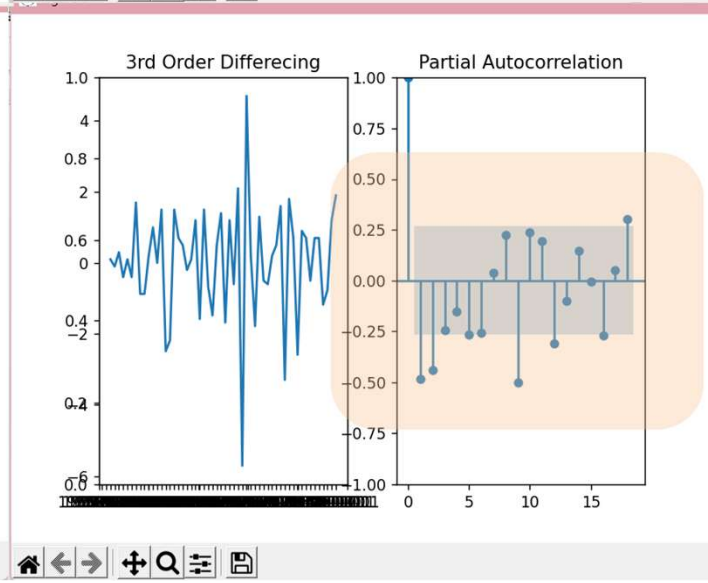


Figure 1
2nd dif
Autocorrelation


3rd dif
Autocorrelation


1st Order Differecing
Partial Autocorrelation


2nd Order Differecing
Partial Autocorrelation


3rd Order Differecing
Partial Autocorrelation

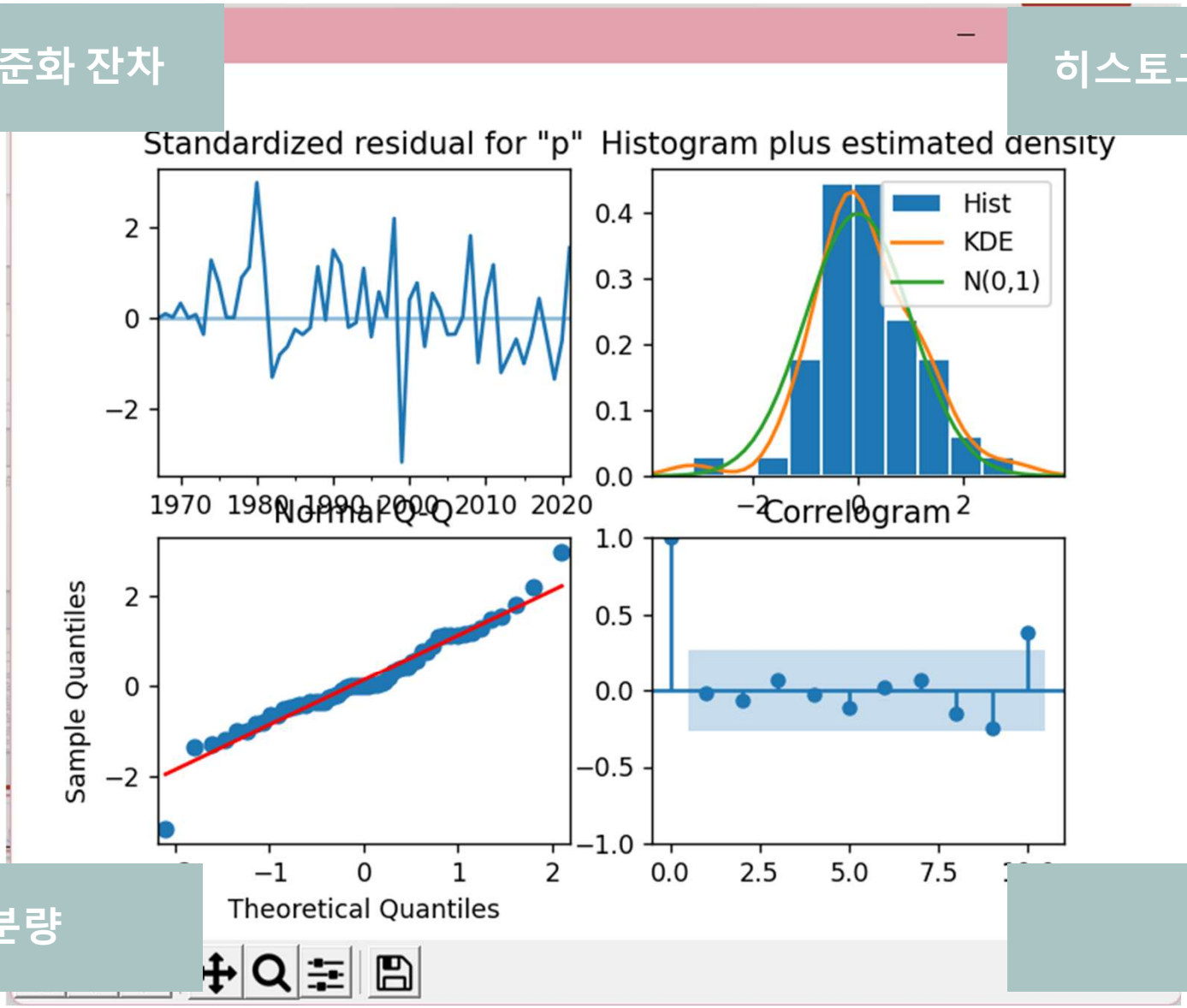
물가 데이터 예측

```python
print('차분 2번한 p-value : ', result_dif2[1])

result_dif3=adfuller(df.price.diff().diff().diff().dropna())
print('차분 3번한 p-value : ', result_dif3[1])

#세번째 차분부터 p값이 확 떨어짐.
#보통 p값은 5%==0.05를 임계값 기준으로 삼는다. (1%, 5%, 10% 중 여기서는 5%를 채택. 데이터 양과 관련.)

#PACF

f=plt.figure()
ax1=f.add_subplot(1,2,1)
ax1.set_title('1st Order Differecing')
dif_1=ax1.plot(df.price.diff())

ax2=f.add_subplot(1,2,2)
plot_pacf(df.price.diff().dropna(), ax=ax2)
plt.show()
```

```python
from statsmodels.tsa.arima.model import ARIMA   # arima.model을 사용해야 함
arima_model=ARIMA(df.price, order=(2,2,1))
# 1,2,1 AIC=156.36 => 이게 가장 값이 낮음
model=arima_model.fit()
print(model.summary())
#Log-likelihood : 가능도, 우도 - 사건이 일어날 가능성
#
model.plot_diagnostics()
plt.show()
startYear=pd.to_datetime('1965-01-01')
endYear=pd.to_datetime('2025-01-01')
predict=model.predict(start=startYear, end=endYear)#2025년도에서 끝날 수 있도록 값 설정.
print(predict)



#csv파일로 만듦
predict.to_csv('predict_price.csv')
```

p에 대한 표준화 잔차

히스토그램 + 추정밀도



이론적 분량

상관도

변동률 데이터

dataProcessing.py | estateIC.py | showPredict.py | estateBS.py | newPrice.py | varianceProcessing.py | m

```python
#세번째로 돌려야할 파일
#예측한 물가지수 데이터를 변동률으로 가공.
#variance.csv가 생성됨

import pandas as pd

vari = pd.read_csv('./predict_price.csv', header=0,names=['years','price'])
vari2 = pd.read_csv('./predict_price.csv', header=0,names=['years','price'])

for i in range(1,61):
    vari2.price[i]=vari.price[i]-vari.price[i-1]
print()
variance=vari2[41:] #인덱스를 열로 삽입
variance.index = variance['years']
variance.set_index('years', inplace=True) #index로 변환
for i in range(0,20):
    (100+variance.price[i])/100
variance.to_csv('variance.csv')
```

newPrice.py predict_price.csv

*.csv files are supported in other JetBrains IDEs

```
1998-01-01,59.23754555724899
1999-01-01,64.76851653398315
2000-01-01,62.81552745480098
2001-01-01,64.96986080724311
2002-01-01,68.08207717094318
2003-01-01,69.38239420726782
2004-01-01,72.19773055982137
2005-01-01,74.73391224705937
2006-01-01,76.42502542325387
2007-01-01,77.96262604328318
2008-01-01,79.99357005619414
2009-01-01,84.81481296982305
2010-01-01,86.00580527096781
2011-01-01,88.79377145897871
2012-01-01,92.91878250044174
2013-01-01,93.78546018877968
2014-01-01,94.63021767346943
2015-01-01,95.835605006391
2016-01-01,96.18167241582816
2017-01-01,97.18902477856518
2018-01-01,99.5060895458344
2019-01-01,100.75304053683496
2020-01-01,100.46069676865231
2021-01-01,101.04235297968559
2022-01-01,104.75818679678643
2023-01-01,106.76389148171204
2024-01-01,108.64544002342102
2025-01-01,110.47393071651759
```

newPrice.py predict_price.csv varianceProcessing.py variance.csv

*.csv files are supported in other JetBrains IDEs

```
years,price
2006-01-01,1.6911131761945057
2007-01-01,1.537600620029309
2008-01-01,2.0309440129109646
2009-01-01,4.8212429136289074
2010-01-01,1.1909923011447603
2011-01-01,2.7879661880108983
2012-01-01,4.125011041463026
2013-01-01,0.8666776883379441
2014-01-01,0.8447574846897652
2015-01-01,1.2053873329215605
2016-01-01,0.3460674094371541
2017-01-01,1.0073523627370236
2018-01-01,2.3170647672692155
2019-01-01,1.2469509910005598
2020-01-01,-0.29234376818263286
2021-01-01,0.5816562110332768
2022-01-01,3.715833817100844
2023-01-01,2.0057046849255897
2024-01-01,1.8815485417089803
2025-01-01,1.8284906930965832
```

부동산 데이터 가공

dataProcessing.py | estateIC.py | showPredict.py | estateBS.py | newPrice.py | varianceProcessing.py | estateProcessing.py | main.py

```python
import numpy as np
import pandas as pd
import seaborn as sns
import warnings

warnings.filterwarnings("ignore")
warnings.filterwarnings(action='ignore')

np.set_printoptions(formatter={'float_kind':'{:f}'.format})
sns.set(rc={'figure.figsize':(8,6)})


data=pd.read_csv("./부동산 지수.csv")   #데이터 삽입

data=data.transpose()   #행 열 전환
data.rename(columns=data.iloc[0], inplace=True)

#null데이터는 버릴거임. ,,,,이 부분들임.
data = data.drop(data.index[0])
data = data.drop(data.index[33:]) #인덱싱할 숫자 값은 보유한 데이터 규격에 맞춰서. 숫자는 엑셀에서 파일 오픈했을 때 마지막 열숫자-1
#print(data)

#csv 데이터의 column name이 전부 영어명이어야함.
#열 하나씩 슬라이싱
Seoul_data = data[['Seoul']]
Gyeonggi_data = data[['Gyeonggi']]
Incheon_data = data[['Incheon']]
Busan_data = data[['Busan']]
Daegu_data = data[['Daegu']]
```

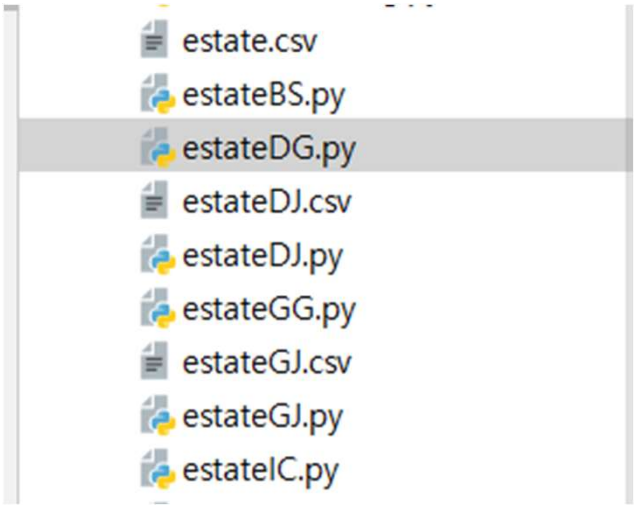
부동산 데이터 가공

```python
Seoul_data = data[['Seoul']]
Gyeonggi_data = data[['Gyeonggi']]
Incheon_data = data[['Incheon']]
Busan_data = data[['Busan']]
Daegu_data = data[['Daegu']]
Gwangju_data = data[['Gwangju']]
Daejeon_data = data[['Daejeon']]
Ulsan_data = data[['Ulsan']]

#인덱스의 값을 열로 넣을 거임. (년도)
#년도는 어차피 모든 데이터가 똑같이 공유하기 때문에 한개만 생성해도 ok
y = Seoul_data.index.values  #csv에서 인덱스 값을 y에 저장
Seoul_data.insert(0, column='years',value=y) #0번째에 years라는 열이름으로 y삽입

y = Gyeonggi_data.index.values  #csv에서 인덱스 값을 y에 저장
Gyeonggi_data.insert(0, column='years',value=y)

y = Incheon_data.index.values  #csv에서 인덱스 값을 y에 저장
Incheon_data.insert(0, column='years',value=y)

y = Busan_data.index.values  #csv에서 인덱스 값을 y에 저장
Busan_data.insert(0, column='years',value=y)

y = Daegu_data.index.values  #csv에서 인덱스 값을 y에 저장
Daegu_data.insert(0, column='years',value=y)

y = Gwangju_data.index.values  #csv에서 인덱스 값을 y에 저장
Gwangju_data.insert(0, column='years',value=y)

y = Daejeon_data.index.values  #csv에서 인덱스 값을 y에 저장
Daejeon_data.insert(0, column='years',value=y)

y = Ulsan_data.index.values  #csv에서 인덱스 값을 y에 저장
Ulsan_data.insert(0, column='years',value=y)



# ##############################데이터가공
#슬라이싱한 열을 각각 하나의 파일로 저장.
Seoul_data.to_csv('estateSL.csv')
Gyeonggi_data.to_csv('estateGG.csv')
Incheon_data.to_csv('estateIC.csv')
Busan_data.to_csv('estateBS.csv')
Daegu_data.to_csv('estateDG.csv')
Gwangju_data.to_csv('estateGJ.csv')
Daejeon_data.to_csv('estateDJ.csv')
Ulsan_data.to_csv('estateUS.csv')
```

newPrice.py | predict_price.csv | varianceProcessing.py | variance.csv | estateProcessing.py | 부동산 지수.csv

*.csv files are supported in other JetBrains IDEs Try RubyMine Try PyCharm Professional

```
,2006-01-01,2006-07-01,2007-01-01,2007-07-01,2008-01-01,2008-07-01,2009-01-01,2009-07-01,2010-01-01,2010-07-01,2011-01-01,2011-0
Seoul,58.59,63.69,71.49,73.08,77.4,80.92,78.5,80.68,81.2,79.75,79.88,79.52,78.14,75.11,73.27,72.8,73.6,74.1,75.53,78.32,79.62,81.19
Gyeonggi,56.07,62.4,71.77,73.33,76.19,78.48,75.23,76.12,75.86,73.79,74.3,74.62,73.94,71.7,69.84,69.77,70.77,71.46,73.25,75.68,76.51
Incheon,52.78,55.09,61.42,65.34,71.76,80.46,79.57,79.78,79.79,78.18,77.83,76.94,75.7,73.34,71.21,71.47,72.99,73.92,75.4,77.52,78.27
Busan,53.04,52.68,52.75,52.92,53.94,55.72,56.33,58.09,61.97,66.45,75.21,82.65,83.61,81.64,80.61,80.43,80.91,81.54,82.84,84.92,86.2,
Daegu,57.38,57.41,56.95,56.01,55.34,54.59,52.59,52.67,53.32,53.73,56.2,60.04,63.82,64.94,67.06,71.22,75.58,77.72,80.97,85.07,85.19,
Gwangju,59.39,60.34,60.77,60.9,61.51,62.58,62.46,62.29,62.75,63.63,68.88,76.94,80.54,81.8,82.44,82.84,83.46,84.29,86.79,90.16,91.17
Daejeon,54.59,53.47,52.76,52.06,52.25,52.59,52.55,54.36,57.14,59.09,64.31,69.9,69.81,68.12,68.32,68.29,68.7,68.74,69.03,69.05,68.96
Ulsan,57.61,62.03,68.03,69.83,71,71.45,70.8,72.36,74.36,75.7,79.53,85.64,91.91,92.92,92.53,92.73,93.67,95.78,97.3,99.5,101.07,101,1
```




```
,years,Daegu
2006-01-01,2006-01-01,57.38
2006-07-01,2006-07-01,57.41
2007-01-01,2007-01-01,56.95
2007-07-01,2007-07-01,56.01
2008-01-01,2008-01-01,55.34
2008-07-01,2008-07-01,54.59
2009-01-01,2009-01-01,52.59
2009-07-01,2009-07-01,52.67
2010-01-01,2010-01-01,53.32
2010-07-01,2010-07-01,53.73
2011-01-01,2011-01-01,56.2
2011-07-01,2011-07-01,60.04
2012-01-01,2012-01-01,63.82
2012-07-01,2012-07-01,64.94
2013-01-01,2013-01-01,67.06
2013-07-01,2013-07-01,71.22
2014-01-01,2014-01-01,75.58
2014-07-01,2014-07-01,77.72
2015-01-01,2015-01-01,80.97
2015-07-01,2015-07-01,85.07
2016-01-01,2016-01-01,85.19
2016-07-01,2016-07-01,83.73
2017-01-01,2017-01-01,82.97
2017-07-01,2017-07-01,83.37
2018-01-01,2018-01-01,84.51
2018-07-01,2018-07-01,85.73
```

*.csv files are supported in other JetBrains IDEs

```
,years,Daejeon
2006-01-01,2006-01-01,54.59
2006-07-01,2006-07-01,53.47
2007-01-01,2007-01-01,52.76
2007-07-01,2007-07-01,52.06
2008-01-01,2008-01-01,52.25
2008-07-01,2008-07-01,52.59
2009-01-01,2009-01-01,52.55
2009-07-01,2009-07-01,54.36
2010-01-01,2010-01-01,57.14
2010-07-01,2010-07-01,59.09
2011-01-01,2011-01-01,64.31
2011-07-01,2011-07-01,69.9
2012-01-01,2012-01-01,69.81
2012-07-01,2012-07-01,68.12
2013-01-01,2013-01-01,68.32
2013-07-01,2013-07-01,68.29
2014-01-01,2014-01-01,68.7
2014-07-01,2014-07-01,68.74
2015-01-01,2015-01-01,69.03
2015-07-01,2015-07-01,69.05
2016-01-01,2016-01-01,68.96
2016-07-01,2016-07-01,69.01
2017-01-01,2017-01-01,69.32
```

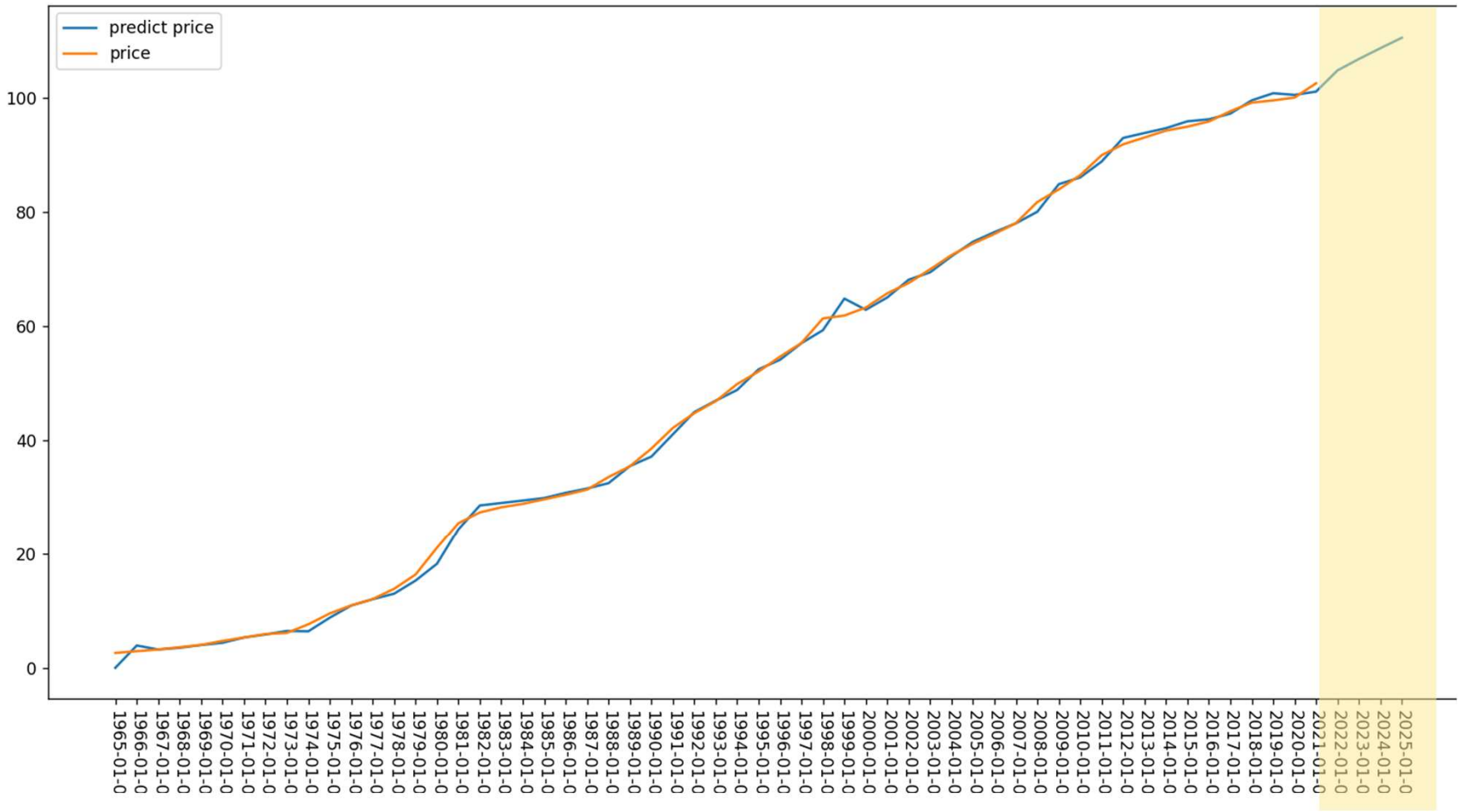
예측 데이터 시각화

dataProcessing.py | estateIC.py | showPredict.py | estateBS.py | newPrice.py | main.py

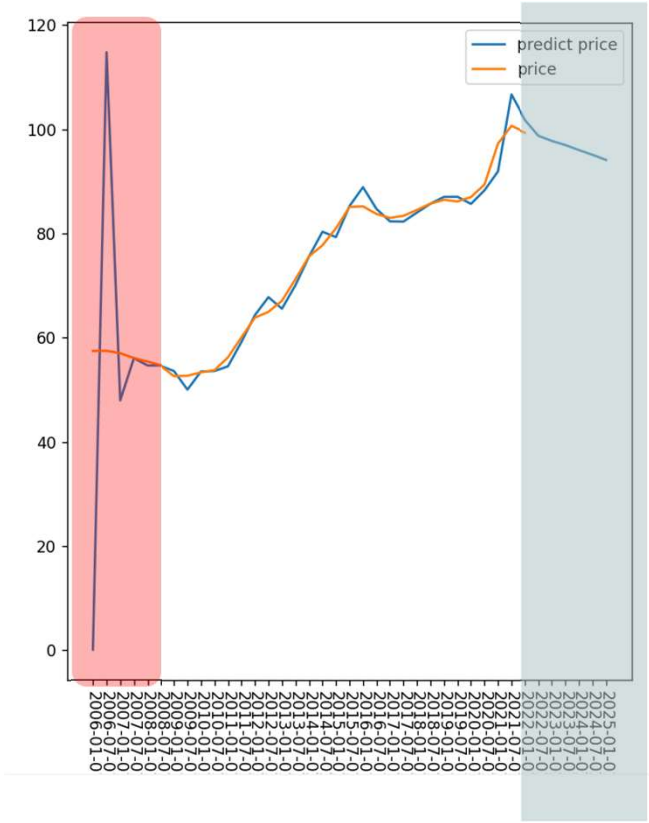
```python
##예측한 파일을 시각화 해줌
##plot_predict메소드가 삭제되어 대체 파일 생성한 것.

import ...

#원본데이터와 예측한 데이터 비교

prePrice = pd.read_csv('./predict_price.csv', header=0,names=['years','pre_price'])
Price=pd.read_csv('./price.csv', header=0, names=['years', 'price'])

# print(prePrice)
print(Price)
# prePrice.index=prePrice['years']


prePrice.index = prePrice['years']
prePrice.set_index('years', inplace=True) #index로 변환
# prePrice['years'] = pd.to_datetime(prePrice['years']) #str to pandas Timestamp


print(prePrice)
print(prePrice.index)


#원형데이터와 예측데이터의 달라진 x축을 맞춰주기 위해서 2025년도까지 none값으로 채워준다.
i = len(Price)
end=len(Price)+4
print(i)
while i <end:
```

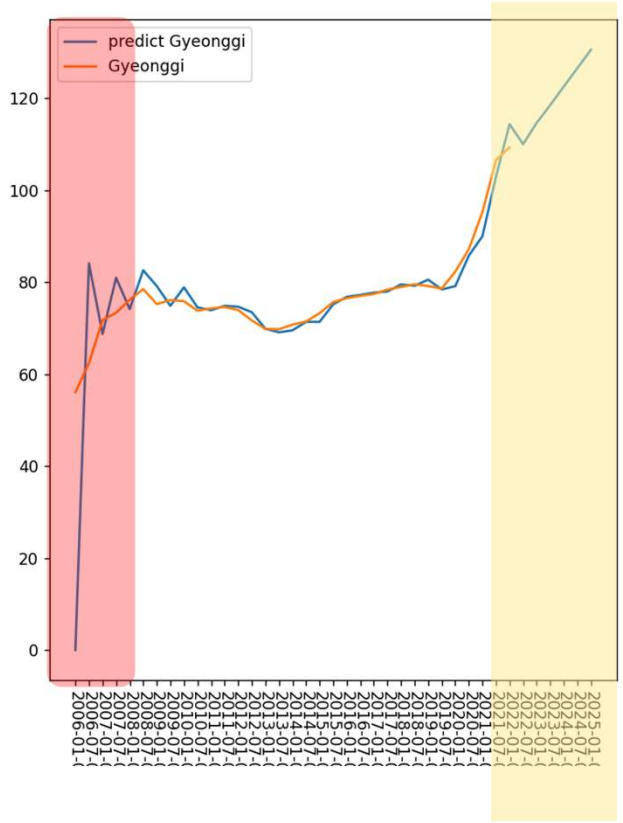
predictBS.py
predictDG.py
predictGG.py
predictIC.py
predictSL.py

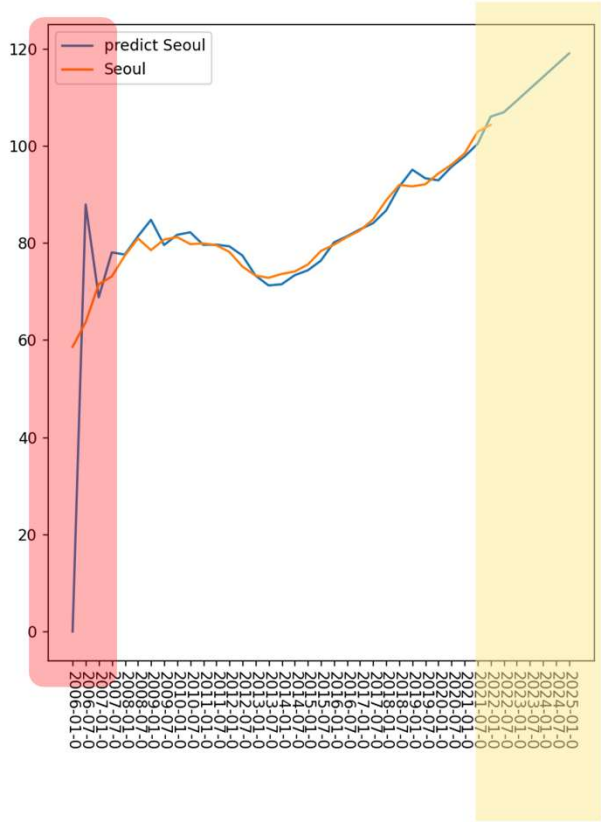
물가 데이터 예측 시각화

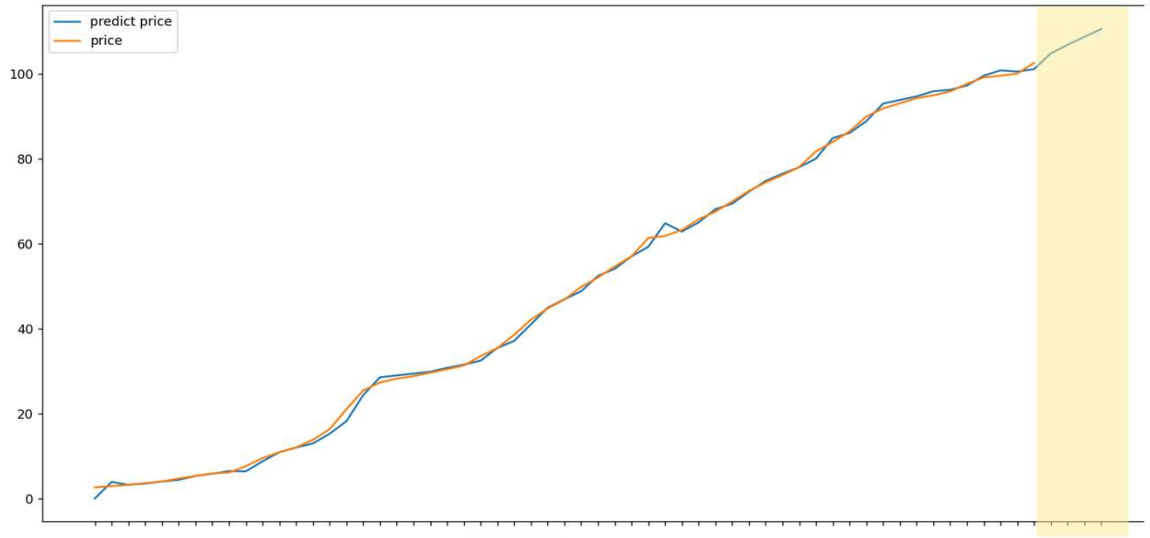
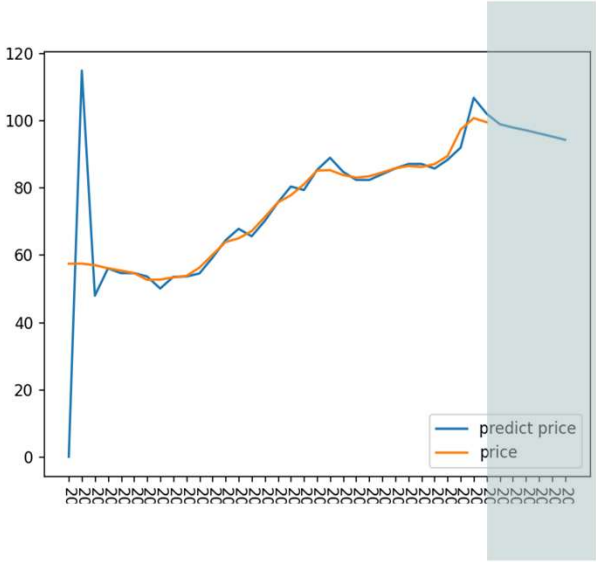
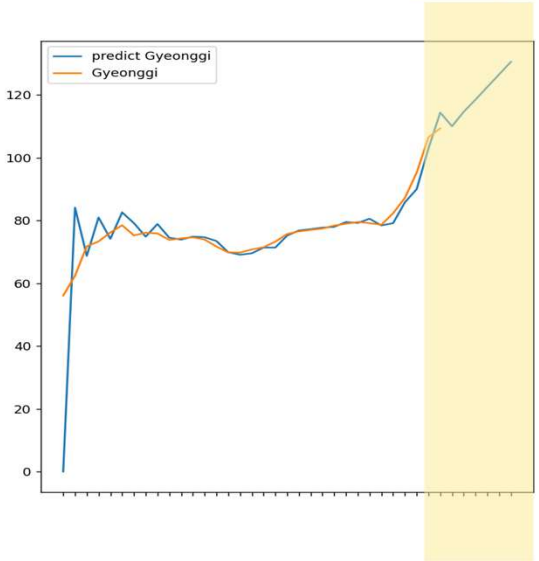

predict price
price
100
80
60
40
20
0
1965-01-0
1966-01-0
1967-01-0
1968-01-0
1969-01-0
1970-01-0
1971-01-0
1972-01-0
1973-01-0
1974-01-0
1975-01-0
1976-01-0
1977-01-0
1978-01-0
1979-01-0
1980-01-0
1981-01-0
1982-01-0
1983-01-0
1984-01-0
1985-01-0
1986-01-0
1987-01-0
1988-01-0
1989-01-0
1990-01-0
1991-01-0
1992-01-0
1993-01-0
1994-01-0
1995-01-0
1996-01-0
1997-01-0
1998-01-0
1999-01-0
2000-01-0
2001-01-0
2002-01-0
2003-01-0
2004-01-0
2005-01-0
2006-01-0
2007-01-0
2008-01-0
2009-01-0
2010-01-0
2011-01-0
2012-01-0
2013-01-0
2014-01-0
2015-01-0
2016-01-0
2017-01-0
2018-01-0
2019-01-0
2020-01-0
2021-01-0
2022-01-0
2023-01-0
2024-01-0
2025-01-0

부동산 데이터 예측 시각화







4-1 분석

가설 입증 및 결론